Chuleta Java — Acceso a Datos (0486 DAM)

Nov 5, 2026 · @Sebas

Índice

1. Mapa rápido: qué va dentro de cada new
2. Imports por tema
   - Plantilla de main
3. Tema 1-2: clase File
4. Tema 3: lectura y escritura secuencial de texto
   - Escribir texto
   - Leer texto
   - Las cuatro formas del FileWriter
5. Tema 4.1: NIO con Path y Files
   - StandardOpenOption: qué poner y cuándo
6. IO vs NIO: equivalencias
7. Tema 4: RandomAccessFile
   - Registros de tamaño fijo
8. Tema 5: binarios y serialización
   - DataOutputStream / DataInputStream
   - ObjectOutputStream / ObjectInputStream
   - Varios objetos en el mismo .dat
9. Tema 6 / 6.1: XML con DOM
   - Leer XML con DOM
   - Crear y guardar XML con DOM
   - XML → ArrayList de objetos (y al revés)
   - Modificar un XML existente
10. Tema 6.2: SAX y DOM vs SAX
11. Tema 7: JAXP, XSD y XPath
    - Validar con XSD
    - XPath
12. Tema 8: JAXB, POJOs y Maven
    - pom.xml de JAXB
    - POJO anotado para JAXB
    - Marshal / unmarshal

    - Comandos Maven
13. Tema 9: buenas prácticas
14. Tema 10: probar y documentar (JUnit 5 + Javadoc)
    - pom.xml para pruebas y Javadoc
    - Clase documentada con Javadoc
    - Clase de prueba con JUnit 5
    - Ejecutar pruebas y generar la documentación
    - Plan de pruebas y qué documentar
15. Tema 12: conversiones entre formatos
    - CSV: leer bien
    - CSV: escribir bien
16. Extra: JSON con Jackson
    - pom.xml de Jackson
    - POJO para Jackson
    - Escribir y leer JSON
    - De JSON a objeto paso a paso
    - Leer varios: JSON → ArrayList<Alumno>
    - Leer JSON sin clase
    - Conversión CSV → JSON

Mapa rápido: qué va dentro de cada new

En java.io las clases se envuelven unas a otras (patrón decorador): la de dentro toca el fichero, las de fuera añaden buffer, líneas, tipos u objetos. La regla: **la de más dentro decide el origen (fichero), la de más fuera decide qué métodos tienes**.

Quiero…	Construcción	Qué aporta cada capa
Escribir texto línea a línea	`new BufferedWriter(new FileWriter(ruta, UTF_8))`	FileWriter abre el fichero · BufferedWriter añade buffer de 8.192 chars y `newLine()`
Escribir texto con println/printf	`new PrintWriter(new BufferedWriter(new FileWriter(ruta, UTF_8)))`	PrintWriter añade `println`, `printf`

Quiero…	Construcción	Qué aporta cada capa
Añadir al final (no machacar)	`new FileWriter(ruta, UTF_8, true)`	3er argumento `append=true` (2º si no hay Charset)
Leer texto línea a línea	`new BufferedReader(new FileReader(ruta, UTF_8))`	BufferedReader aporta `readLine()` (FileReader solo tiene `read()` → int)
Leer/escribir texto desde bytes (antes de Java 11)	`new BufferedReader(new InputStreamReader(new FileInputStream(ruta), UTF_8))`	InputStreamReader es el puente bytes → caracteres con Charset
Leer texto con NIO	`Files.newBufferedReader(path, UTF_8)`	Devuelve el mismo BufferedReader, sin encadenar news
Escribir texto con NIO	`Files.newBufferedWriter(path, UTF_8)`	Igual, pero BufferedWriter
Tipos primitivos en binario	`new DataOutputStream(new BufferedOutputStream(new FileOutputStream(ruta)))`	FileOutputStream escribe bytes · Buffered acelera · Data añade `writeInt`, `writeUTF`, `writeDouble`
Leer primitivos	`new DataInputStream(new BufferedInputStream(new FileInputStream(ruta)))`	`readInt`, `readUTF` … en el MISMO orden en que se escribió
Guardar objetos	`new ObjectOutputStream(new FileOutputStream(ruta))`	ObjectOutputStream añade `writeObject` (la clase debe ser Serializable)
Leer objetos	`new ObjectInputStream(new FileInputStream(ruta))`	`readObject()` + cast
Acceso aleatorio	`new RandomAccessFile(ruta, "rw")`	No se envuelve en nada: lee y escribe, con `seek()`

Bytes vs caracteres. `InputStream` / `OutputStream` (y todo lo que acaba en *Stream*) = bytes: binario, imágenes, objetos. `Reader` / `Writer` = texto, y por eso llevan Charset. Nunca mezcles: un Reader no va dentro de un Stream ni al revés, salvo con el puente `InputStreamReader` / `OutputStreamWriter`.

Siempre: Charset explícito (`StandardCharsets.UTF_8`), carpeta creada antes (`Files.createDirectories`) y try-with-resources (cerrar = volcar el buffer).

Imports por tema y plantilla de main

Si no hay autocompletado, copia el bloque del tema que toque. Con `.*` no tienes que acordarte de cada clase.

```
// Ficheros (temas 2-5, 9, 12)
import java.io.*;                          // File, FileReader/Writer,
Buffered*, Data*, Object*, RandomAccessFile
import java.nio.charset.StandardCharsets;
import java.nio.file.*;                    // Path, Files, StandardOpenOption,
StandardCopyOption, NoSuchFileException
import java.util.*;                        // List, ArrayList, Arrays, Scanner
import java.util.stream.Stream;            // Files.lines / walk / list

// XML con DOM / SAX / XPath / XSD (temas 6-7)
import javax.xml.XMLConstants;
import javax.xml.parsers.*;                // DocumentBuilderFactory,
DocumentBuilder, SAXParserFactory, SAXParser
import javax.xml.transform.*;              // TransformerFactory, Transformer,
OutputKeys
import javax.xml.transform.dom.DOMSource;
import javax.xml.transform.stream.StreamResult;
import javax.xml.xpath.*;                  // XPathFactory, XPath,
XPathConstants, XPathExpression
import javax.xml.validation.*;             // SchemaFactory, Schema
import org.w3c.dom.*;                      // Document, Element, Node,
NodeList
import org.xml.sax.*;                      // Attributes, ErrorHandler,
SAXException, SAXParseException
import org.xml.sax.helpers.DefaultHandler;

// JAXB (tema 8)
import jakarta.xml.bind.*;                 // JAXBContext, Marshaller,
Unmarshaller, JAXBException
import jakarta.xml.bind.annotation.*;      // @XmlRootElement, @XmlElement...

// Jackson (extra)
import com.fasterxml.jackson.databind.*;  // ObjectMapper,
SerializationFeature, JsonNode
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.annotation.*;
```

Comprobado: todos los imports del JDK de arriba juntos compilan sin ambigüedades. Si alguna vez el compilador dice *reference to X is ambiguous* (dos paquetes con una clase del mismo nombre), importa esa clase sola, por ejemplo `import org.w3c.dom.Element;`.

Plantilla de main

Las excepciones van de la más concreta a la más general; al revés no compila:

```
public class Main {
    public static void main(String[] args) {
        try {
            Files.createDirectories(Path.of("datos"));
            // ... tu código ...
        } catch (FileNotFoundException | NoSuchFileException e) {   //
fichero o carpeta que no existe
            System.out.println("No existe: " + e.getMessage());
        } catch (IOException e) {                                    // resto
de E/S (incluye EOF, StreamCorrupted)
            System.out.println("Error de E/S: " + e);
        } catch (ClassNotFoundException e) {                         //
readObject
            System.out.println("Clase no encontrada: " + e);
        } catch (ParserConfigurationException | SAXException |
TransformerException e) { // XML
            System.out.println("Error XML: " + e);
        } catch (XPathExpressionException e) {                       // XPath
mal escrito
            System.out.println("XPath: " + e);
        }
        // JAXB: catch (JAXBException e)  ·  Jackson: sus excepciones ya son
IOException
    }
}
```

Java solo deja capturar una excepción comprobada si el `try` puede lanzarla: quita los `catch` de lo que no uses (por ejemplo, `ClassNotFoundException` sin `readObject`). Si vas justo de tiempo, `public static void main(String[] args) throws Exception` compila siempre.

Tema 1-2: clase File (java.io)

`new File(...)` solo representa una ruta: no crea, no comprueba y no falla. Lo que hace algo son sus métodos.

```
File archivo = new File("datos/ejemplo.txt");
File otro    = new File(carpeta, "fichero.txt"); // portable: padre + hijo

archivo.exists();          // ¿existe?
archivo.createNewFile();   // crea el fichero vacío (false si ya existía). La
```

```
carpeta debe existir
archivo.delete();          // borra (carpeta solo si está vacía)
archivo.length();          // tamaño en bytes
archivo.getName();         // "ejemplo.txt"
archivo.getAbsolutePath(); // ruta completa
archivo.isDirectory(); archivo.isFile();
archivo.renameTo(new File("datos/nuevo.txt"));

File carpeta = new File("datos/pruebas");
carpeta.mkdir();   // crea SOLO si el padre existe
carpeta.mkdirs();  // crea la carpeta y todos los padres que falten

String[] nombres = carpeta.list();     // solo nombres
File[]   lista   = carpeta.listFiles(); // objetos File (puedes preguntar
isDirectory)
for (File f : lista)
    System.out.println(f.getName() + (f.isDirectory() ? " (dir)" : ""));
```

Similar	Diferencia
`mkdir()` vs `mkdirs()`	mkdir falla (devuelve false) si falta un padre; mkdirs crea toda la cadena
`list()` vs `listFiles()`	`String[]` con nombres vs `File[]` con objetos completos
`createNewFile()` vs `Files.createFile(path)`	boolean silencioso vs excepción que explica el fallo (`FileAlreadyExistsException`)
`mkdirs()` vs `Files.createDirectories(path)`	boolean vs excepción con causa; los dos crean padres y no fallan si ya existe

Orden correcto: primero `Files.createDirectories(Path.of("datos"))` , después `createNewFile()` .

Tema 3: lectura y escritura secuencial de texto (java.io)

Secuencial = del principio al final, en orden. Para texto: Reader/Writer + buffer.

Escribir texto

```
Files.createDirectories(Path.of("datos"));          // la carpeta NO se crea
sola
try (BufferedWriter bw = new BufferedWriter(
        new FileWriter("datos/registro.txt", StandardCharsets.UTF_8))) {
    bw.write("Registro 1");   // no añade salto de línea
    bw.newLine();              // separador del sistema (\r\n Win, \n Linux)
}   // al cerrar se hace flush automático
```

Leer texto

```
try (BufferedReader br = new BufferedReader(
        new FileReader("datos/registro.txt", StandardCharsets.UTF_8))) {
    String linea;
    while ((linea = br.readLine()) != null) {   // null = fin de fichero
        System.out.println(linea);
    }
} catch (FileNotFoundException e) { ... }     // primero la concreta
  catch (IOException e) { ... }
```

Las cuatro formas del FileWriter

```
new FileWriter("d.txt");                                  // machaca, charset por
defecto (evitar)
new FileWriter("d.txt", true);                            // añade, charset por
defecto (evitar)
new FileWriter("d.txt", StandardCharsets.UTF_8);    // machaca, UTF-8
new FileWriter("d.txt", StandardCharsets.UTF_8, true); // añade, UTF-8
```

Clase / método	Para qué	Diferencia con su similar
`FileWriter`	Conecta con el fichero y escribe caracteres	Sin buffer: cada write va al SO (lento)
`BufferedWriter`	Acumula en memoria y vuelca de golpe; `newLine()`	10–100 veces más rápido que FileWriter solo
`PrintWriter`	`println`, `printf`, `print`	Comodidad; se pone encima del BufferedWriter
`FileReader`	Abre para leer; `read()` devuelve un char como int, -1 al final	No sabe qué es una línea

Clase / método	Para qué	Diferencia con su similar
`BufferedReader`	Buffer + `readLine()`	`readLine()` quita el salto y devuelve null al final
`Scanner`	`new Scanner(fichero, UTF_8)`; `nextLine()`, `nextInt()`	Parsea tokens; más lento que BufferedReader
`flush()` vs `close()`	flush vuelca sin cerrar; close vuelca y cierra	try-with-resources llama a close por ti
`write("\n")` vs `newLine()`	"\n" fijo vs separador del SO	newLine es portable

Trampas: sin cerrar un BufferedWriter el fichero queda vacío (0 bytes) sin error. Escribir y leer con Charsets distintos da mojibake (`lÃnea`). Desde Java 18 el charset por defecto es UTF-8, antes en Windows era windows-1252: por eso siempre explícito. El constructor `FileWriter(ruta, Charset)` existe desde Java 11; antes se usaba `new OutputStreamWriter(new FileOutputStream(ruta), UTF_8)`.

Tema 4.1: NIO con Path y Files (java.nio.file)

`Path` es la ruta (sustituye a File) y `Files` es una clase de métodos estáticos que hace el trabajo. Muchos métodos abren, escriben y cierran en una sola llamada.

```
Path ruta = Path.of("datos", "nio_prueba.txt");   // Java 11+; Paths.get(...)
hace lo mismo
Files.createDirectories(ruta.getParent());          // ojo: getParent() es
null si la ruta no tiene carpeta

// Comprobar / crear / borrar / copiar
Files.exists(ruta);
Files.createFile(ruta);        // excepción si ya existe
Files.createDirectory(dir);    // excepción si ya existe o falta el padre
Files.createDirectories(dir);  // crea padres, no falla si ya existe
Files.delete(ruta);            // excepción si no existe
Files.deleteIfExists(ruta);
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
Files.move(origen, destino);
Files.size(ruta);

// Texto de golpe (ficheros pequeños)
Files.writeString(ruta, "Hola\n", StandardCharsets.UTF_8);
String todo = Files.readString(ruta, StandardCharsets.UTF_8);
Files.write(ruta, List.of("línea 1", "línea 2"), StandardCharsets.UTF_8);
```

```
List<String> lineas = Files.readAllLines(ruta, StandardCharsets.UTF_8);

// Añadir al final (opciones al final de la llamada)
Files.write(ruta, List.of("Cuarta"), StandardCharsets.UTF_8,
            StandardOpenOption.CREATE, StandardOpenOption.APPEND);

// Ficheros grandes: línea a línea
try (BufferedReader br = Files.newBufferedReader(ruta,
StandardCharsets.UTF_8)) { ... }
try (BufferedWriter bw = Files.newBufferedWriter(ruta,
StandardCharsets.UTF_8)) { ... }
try (Stream<String> s = Files.lines(ruta, StandardCharsets.UTF_8)) {   //
stream perezoso
    s.filter(l -> l.contains("x")).forEach(System.out::println);
}

// Recorrer carpetas
try (Stream<Path> s = Files.list(dir)) { ... }   // un nivel
try (Stream<Path> s = Files.walk(dir)) { ... }   // recursivo

// Canal de bytes (bajo nivel)
try (SeekableByteChannel canal = Files.newByteChannel(ruta,
        StandardOpenOption.CREATE, StandardOpenOption.WRITE,
        StandardOpenOption.TRUNCATE_EXISTING)) { ... }
```

Método	Cuándo	Ojo
`readString` / `writeString`	Fichero pequeño, lo quiero entero en un String	Carga todo en memoria
`readAllLines` / `write(lista)`	Fichero pequeño, por líneas	No es más eficiente: también carga todo
`lines`	Fichero grande, procesar con streams	Usar en try-with-resources
`newBufferedReader/Writer`	Fichero grande, bucle clásico	Mismo BufferedReader del tema 3
`write` / `writeString`	Por defecto truncan (machacan)	Para añadir: `CREATE, APPEND`
`newByteChannel` con WRITE+CREATE	Escritura en bytes	Sin `TRUNCATE_EXISTING` quedan restos del contenido viejo

StandardOpenOption: qué poner y cuándo

Son las opciones que van al final de `Files.write`, `writeString`, `newBufferedWriter`, `newOutputStream` y `newByteChannel`. Se importan con `import java.nio.file.StandardOpenOption;` (o `import static java.nio.file.StandardOpenOption.*;` para escribir solo `CREATE, APPEND`). **Si no pones ninguna**, escribir equivale a `CREATE, TRUNCATE_EXISTING, WRITE` (crea o machaca) y leer equivale a `READ`.

Quiero…	Opciones	Si no existe	Si ya existe
Crear o machacar (lo normal)	ninguna	Lo crea	Lo vacía y escribe
Añadir al final	`CREATE, APPEND`	Lo crea	Añade detrás
Añadir solo si ya existe	`APPEND`	`NoSuchFileException`	Añade detrás
Crear solo si es nuevo (no pisar nada)	`CREATE_NEW`	Lo crea	`FileAlreadyExistsException`
Escribir en un fichero que debe existir	`WRITE, TRUNCATE_EXISTING`	`NoSuchFileException`	Lo vacía y escribe
Fichero temporal	`CREATE, WRITE, DELETE_ON_CLOSE`	Lo crea	Se borra al cerrar
Que cada escritura llegue al disco ya	añadir `SYNC` (datos + metadatos) o `DSYNC` (solo datos)	—	Más lento

Opción	Qué hace
`READ`	Abrir para leer (por defecto en lecturas)
`WRITE`	Abrir para escribir; no crea ni vacía por sí sola
`CREATE`	Crea el fichero si no existe; si existe, lo abre tal cual
`CREATE_NEW`	Crea el fichero; falla si ya existe
`APPEND`	Escribe al final
`TRUNCATE_EXISTING`	Vacía el fichero al abrirlo
`DELETE_ON_CLOSE`	Borra el fichero al cerrar

Opción	Qué hace
`SYNC` / `DSYNC`	Escritura síncrona en disco

```
import static java.nio.file.StandardOpenOption.*;

Path p = Path.of("datos", "log.txt");
Files.writeString(p, "inicio\n", StandardCharsets.UTF_8);                //
crea o machaca
Files.writeString(p, "línea\n", StandardCharsets.UTF_8, CREATE, APPEND); //
añade
Files.write(p, List.of("L1", "L2"), StandardCharsets.UTF_8, CREATE, APPEND);
try (BufferedWriter bw = Files.newBufferedWriter(p, StandardCharsets.UTF_8,
CREATE, APPEND)) {
    bw.write("con buffer"); bw.newLine();
}
try (OutputStream out = Files.newOutputStream(p, CREATE, APPEND)) { ... }
// bytes
```

Trampas (comprobadas en JDK 25):

- `APPEND` sin `CREATE` sobre un fichero que no existe lanza `NoSuchFileException`. Ponlos siempre juntos.
- `CREATE` sin `TRUNCATE_EXISTING` ni `APPEND` no vacía el fichero: escribe desde el principio encima de lo que había. Con `"dos\ntres\n"` y escribiendo `"X\n"` queda `"X\ns\ntres\n"`.
- `APPEND` junto con `TRUNCATE_EXISTING`, o `READ` en un método de escritura, lanza `IllegalArgumentException`.
- En cuanto pones alguna opción, dejas de tener las de por defecto: `Files.writeString(p, s, UTF_8, WRITE)` ya no crea el fichero.

Equivalencias con java.io: `new FileWriter(f, UTF_8)` = sin opciones; `new FileWriter(f, UTF_8, true)` = `CREATE, APPEND`.

IO vs NIO: equivalencias y cuándo usar cada uno

Usa NIO (`Path` + `Files`) por defecto en código nuevo; usa java.io cuando necesites las clases de flujo que NIO no sustituye (Data/Object streams, RandomAccessFile) o cuando el ejercicio lo pida. Se combinan: `Files.newInputStream(path)` te da un InputStream para envolver.

Tarea	java.io	java.nio.file
Representar ruta	`new File("datos/a.txt")`	`Path.of("datos", "a.txt")`
Ruta padre + hijo	`new File(carpeta, "a.txt")`	`carpeta.resolve("a.txt")`
¿Existe?	`f.exists()`	`Files.exists(p)`
Crear fichero	`f.createNewFile()` → boolean	`Files.createFile(p)` → excepción si falla
Crear carpetas	`f.mkdirs()`	`Files.createDirectories(p)`
Borrar	`f.delete()` → boolean	`Files.delete(p)` / `deleteIfExists(p)`
Listar	`f.listFiles()`	`Files.list(p)` / `Files.walk(p)`
Tamaño	`f.length()`	`Files.size(p)`
Leer línea a línea	`new BufferedReader(new FileReader(f, UTF_8))`	`Files.newBufferedReader(p, UTF_8)`
Escribir línea a línea	`new BufferedWriter(new FileWriter(f, UTF_8))`	`Files.newBufferedWriter(p, UTF_8)`
Añadir	`new FileWriter(f, UTF_8, true)`	`Files.write(p, l, UTF_8, CREATE, APPEND)`
Leer todo	bucle a mano	`Files.readString` / `readAllLines`
Bytes	`new FileInputStream(f)`	`Files.newInputStream(p)` / `readAllBytes(p)`
Convertir	`f.toPath()`	`p.toFile()`

Diferencias clave: java.io devuelve booleans silenciosos y depende de la codificación por defecto; NIO lanza excepciones que explican el fallo (`NoSuchFileException` , `FileAlreadyExistsException`), pide el Charset explícito, construye rutas portables y ofrece streams (`lines` , `walk`). RandomAccessFile y ObjectStreams siguen siendo de java.io.

Tema 4: acceso aleatorio con RandomAccessFile

Saltar a cualquier byte sin leer lo anterior. No se envuelve en nada: lee y escribe a la vez. Útil con registros de tamaño fijo (registro n en la posición `n * TAM`).

```
Files.createDirectories(Path.of("datos"));
try (RandomAccessFile raf = new RandomAccessFile("datos/registros.dat",
"rw")) {
    raf.writeUTF("Registro 1");   // 2 bytes de longitud + el texto
    raf.writeUTF("Registro 2");
    raf.seek(0);                   // volver al inicio
    System.out.println(raf.readUTF());
    raf.skipBytes(2 + "Registro 2".length()); // saltar uno
    raf.getFilePointer();          // posición actual en bytes
    raf.length();                  // tamaño total
}
```

Elemento	Para qué
Modo `"r"`	Solo lectura (excepción si no existe)
Modo `"rw"`	Lectura y escritura; crea el fichero si no existe
`seek(pos)`	Mueve el puntero al byte `pos`
`getFilePointer()`	Dice dónde está el puntero
`skipBytes(n)`	Avanza n bytes desde la posición actual
`writeInt/readInt`, `writeDouble/readDouble`, `writeUTF/readUTF`	Tipos primitivos (int = 4 bytes, double = 8, UTF = 2 + longitud)

vs secuencial: BufferedReader solo avanza hacia delante; RandomAccessFile va a cualquier posición y modifica en el sitio. `writeUTF` no es un fichero de texto legible: para texto usa `Files.write(..., UTF_8)`.

Registros de tamaño fijo (el ejercicio típico)

Si todos los registros ocupan lo mismo, el registro n empieza en `n * TAM` y puedes ir directo a él, modificarlo o borrarlo sin tocar el resto. Por eso el `String` se escribe con longitud fija (`writeChars`), nunca con `writeUTF` (ocupa lo que mida el texto).

```
static final int TAM_NOMBRE = 20;                // caracteres fijos del
nombre
static final int TAM = 4 + TAM_NOMBRE * 2 + 8;   // int(4) + 20 chars(2
bytes c/u) + double(8) = 52

static void escribirRegistro(RandomAccessFile raf, long pos, int id, String
nombre, double nota) throws IOException {
```

```
    raf.seek(pos * TAM);
    raf.writeInt(id);
    StringBuilder sb = new StringBuilder(nombre);
    sb.setLength(TAM_NOMBRE);                    // corta si es largo,
rellena con '\0' si es corto
    raf.writeChars(sb.toString());
    raf.writeDouble(nota);
}

static String leerRegistro(RandomAccessFile raf, long pos) throws IOException
{
    raf.seek(pos * TAM);
    int id = raf.readInt();
    char[] c = new char[TAM_NOMBRE];
    for (int i = 0; i < TAM_NOMBRE; i++) c[i] = raf.readChar();
    String nombre = new String(c).trim();        // trim quita los '\0' del
relleno
    double nota = raf.readDouble();
    return id + " | " + nombre + " | " + nota;
}

try (RandomAccessFile raf = new RandomAccessFile("datos/alumnos.dat", "rw"))
{
    escribirRegistro(raf, 0, 1, "Ana Torres", 8.5);
    escribirRegistro(raf, 1, 2, "Luis Pérez", 6.0);
    escribirRegistro(raf, 2, 3, "Marta Gómez Fernández de la Vega", 7.25); //
se corta a 20

    escribirRegistro(raf, raf.length() / TAM, 4, "Pablo Ruiz", 5.0); //
AÑADIR al final

    raf.seek(1 * TAM + 4 + TAM_NOMBRE * 2);      // MODIFICAR solo la nota
del registro 1
    raf.writeDouble(9.5);

    raf.seek(2 * TAM);                           // BORRADO LÓGICO del
registro 2
    raf.writeInt(-1);

    long n = raf.length() / TAM;                 // nº de registros
    for (long i = 0; i < n; i++) {
        raf.seek(i * TAM);
        if (raf.readInt() == -1) continue;       // saltar borrados
        System.out.println(leerRegistro(raf, i));
    }
}
```

Salida comprobada (Java 21 y Java 25): `Registros: 4 (208 bytes)`, luego `1 | Ana Torres | 8.5`, `2 | Luis Pérez | 9.5`, `4 | Pablo Ruiz | 5.0`.

Operación	Cómo
Ir al registro n	`raf.seek(n * TAM)`
Campo dentro del registro	`raf.seek(n * TAM + bytes de los campos anteriores)`
Número de registros	`raf.length() / TAM`
Añadir (no hay modo append)	`raf.seek(raf.length())` y escribir
Vaciar el fichero	`raf.setLength(0)`
Borrado lógico	Marcar el id con -1 y saltarlo al leer
Tamaños	int 4 · long 8 · double 8 · float 4 · char 2 · boolean 1

Ojo: `"rw"` no vacía el fichero al abrir. El puntero empieza en 0, así que escribir sin `seek` sobrescribe el principio y deja el resto como estaba.

Tema 5: ficheros binarios y serialización

Dos niveles: DataStreams guardan primitivos campo a campo; ObjectStreams guardan el objeto entero.

Primitivos con DataOutputStream / DataInputStream

```
try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(Files.newOutputStream(ruta)))) {  // o new
FileOutputStream(...)
    out.writeInt(lista.size());      // cabecera: cuántos vienen
    for (Alumno a : lista) {
        out.writeInt(a.id()); out.writeUTF(a.nombre());
out.writeDouble(a.nota());
    }
}
try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(Files.newInputStream(ruta)))) {
    int n = in.readInt();
    for (int i = 0; i < n; i++)      // MISMO orden que al escribir
        lista.add(new Alumno(in.readInt(), in.readUTF(), in.readDouble()));
}
```

Objetos con ObjectOutputStream / ObjectInputStream

```
public class Producto implements Serializable {
    private static final long serialVersionUID = 1L;   // versión de la clase
    private String nombre;
    private double precio;
    private transient String cache;                    // NO se guarda
(vuelve null/0)
    public Producto(String nombre, double precio) { ... }
    // Serializable NO necesita constructor vacío (a diferencia de JAXB y
Jackson)
}

try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream("datos/producto.dat"))) {
    oos.writeObject(new Producto("Teclado", 29.99));
}
try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream("datos/producto.dat"))) {
    Producto p = (Producto) ois.readObject();          // cast obligatorio
} catch (IOException | ClassNotFoundException e) { ... }
```

Concepto	Qué es / diferencia
`FileOutputStream`	Escribe bytes en disco; va siempre DENTRO
`DataOutputStream`	Primitivos (`writeInt` , `writeUTF` …); tú decides el formato y el orden
`ObjectOutputStream`	Objetos completos con `writeObject` ; formato propio de Java
`Serializable`	Interfaz marcadora: sin ella, `NotSerializableException` . Los campos que sean objetos también deben serlo
`transient`	Campo que no se serializa
`serialVersionUID`	Si cambia la clase y no coincide → `InvalidClassException`
`ClassNotFoundException`	Se captura en readObject: el .dat guarda el nombre de la clase
Data vs Object	Data: compacto y legible desde otro lenguaje si conoces el formato. Object: cómodo pero solo Java y frágil ante cambios de clase

Varios objetos en el mismo .dat

Dos formas. La A escribe objeto a objeto y se lee en bucle hasta que salta `EOFException` (es la forma normal de saber que se acabó). La B guarda la lista entera con un solo `writeObject` , y es la más sencilla.

```
// A) Uno a uno
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("datos/productos.dat"))) {
    oos.writeObject(new Producto("Teclado", 29.99));
    oos.writeObject(new Producto("Ratón", 15.5));
}
List<Producto> leidos = new ArrayList<>();
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("datos/productos.dat"))) {
    while (true) {
        leidos.add((Producto) ois.readObject());
    }
} catch (EOFException fin) {
    // fin del fichero: salida normal del bucle
}
```

```
// B) La lista entera de una vez (ArrayList ya es Serializable)
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("datos/lista.dat"))) {
    oos.writeObject(lista);
}
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("datos/lista.dat"))) {
    @SuppressWarnings("unchecked")
    ArrayList<Producto> l = (ArrayList<Producto>) ois.readObject();
}
```

Trampa: añadir a un .dat que ya existe. Cada `new ObjectOutputStream` escribe una cabecera. Si abres con `new FileOutputStream(f, true)` y añades, al leer aparece una segunda cabecera en medio y salta `StreamCorruptedException: invalid type code: AC` (comprobado). Hay dos soluciones:

1. Leer todo a una lista, añadir el nuevo y reescribir el fichero entero (forma B). Es la más segura.
2. Una subclase que no escriba la cabecera cuando el fichero ya tiene datos:

```
class AppendObjectOutputStream extends ObjectOutputStream {
    AppendObjectOutputStream(OutputStream out) throws IOException {
super(out); }
    @Override protected void writeStreamHeader() throws IOException {
reset(); }
}

File f = new File("datos/productos.dat");
try (ObjectOutputStream oos = f.exists() && f.length() > 0
        ? new AppendObjectOutputStream(new FileOutputStream(f, true))   // ya
hay cabecera
        : new ObjectOutputStream(new FileOutputStream(f))) {            //
fichero nuevo
    oos.writeObject(new Producto("Monitor", 199.0));
}
```

Tema 6 / 6.1: XML con DOM

DOM carga el XML entero en memoria como un árbol que puedes recorrer y modificar. Cadena: Factory → Builder → Document.

Leer XML con DOM

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
// seguridad
dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);
// anti XXE
DocumentBuilder db = dbf.newDocumentBuilder();
Document doc = db.parse(new File("datos/empleados.xml"));
doc.getDocumentElement().normalize();                       // opcional: une
textos partidos

NodeList lista = doc.getElementsByTagName("empleado");
for (int i = 0; i < lista.getLength(); i++) {
    Element emp = (Element) lista.item(i);
    String id     = emp.getAttribute("id");                 // atributo
    String nombre =
emp.getElementsByTagName("nombre").item(0).getTextContent(); // hijo
}
```

Crear y guardar XML con DOM

```
Document doc = dbf.newDocumentBuilder().newDocument();
Element root = doc.createElement("empleados");
doc.appendChild(root);
Element emp = doc.createElement("empleado");
emp.setAttribute("id", "E001");
Element nombre = doc.createElement("nombre");
nombre.appendChild(doc.createTextNode("Ana Torres"));      // o
nombre.setTextContent(...)
emp.appendChild(nombre);
root.appendChild(emp);

TransformerFactory tf = TransformerFactory.newInstance();
tf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
Transformer t = tf.newTransformer();
t.setOutputProperty(OutputKeys.INDENT, "yes");
t.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
t.transform(new DOMSource(doc), new StreamResult(new
File("datos/salida.xml")));
```

Método / clase	Para qué
`DocumentBuilderFactory`	Crea parsers DOM (no analiza nada aún)

Método / clase	Para qué
`DocumentBuilder.parse()` / `newDocument()`	Leer un XML / empezar uno vacío
`getElementsByTagName`	En `doc` busca en todo; en un `Element` solo dentro de él
`getAttribute` vs `getTextContent`	Lo de dentro de la etiqueta (`id=""`) vs el texto entre etiquetas
`createElement` + `appendChild`	Crear nodo y colgarlo del padre (sin append no aparece)
`setTextContent` vs `createTextNode`	Atajo vs crear el nodo de texto explícito; mismo resultado
`removeChild`	Quitar un nodo: `padre.removeChild(hijo)`
`Transformer` + `DOMSource` + `StreamResult`	Volcar el árbol a fichero (DOM no sabe escribirse solo)

XML → ArrayList de objetos (y al revés)

Es el paso previo a cualquier conversión: del XML a una lista de objetos y de la lista a XML. Los métodos `builder()` y `guardar()` se reutilizan en todo lo demás.

```
record Empleado(String id, String nombre, String puesto) {}   // o una clase
con getters

static DocumentBuilder builder() throws ParserConfigurationException {
    DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
    dbf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
    dbf.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
true);
    return dbf.newDocumentBuilder();
}

static ArrayList<Empleado> leerXml(Path ruta) throws Exception {
    Document doc = builder().parse(ruta.toFile());
    ArrayList<Empleado> lista = new ArrayList<>();
    NodeList nodos = doc.getElementsByTagName("empleado");
    for (int i = 0; i < nodos.getLength(); i++) {
        Element emp = (Element) nodos.item(i);
        lista.add(new Empleado(
            emp.getAttribute("id"),
            emp.getElementsByTagName("nombre").item(0).getTextContent(),
```

```
            emp.getElementsByTagName("puesto").item(0).getTextContent()));
    }
    return lista;
}

static void escribirXml(List<Empleado> lista, Path ruta) throws Exception {
    Document doc = builder().newDocument();
    Element root = doc.createElement("empleados");
    doc.appendChild(root);
    for (Empleado e : lista) {
        Element emp = doc.createElement("empleado");
        emp.setAttribute("id", e.id());
        Element nombre = doc.createElement("nombre");
        nombre.setTextContent(e.nombre());
        Element puesto = doc.createElement("puesto");
        puesto.setTextContent(e.puesto());
        emp.appendChild(nombre);
        emp.appendChild(puesto);
        root.appendChild(emp);
    }
    guardar(doc, ruta);
}
```

Modificar un XML existente

```
static void modificarXml(Path ruta) throws Exception {
    Document doc = builder().parse(ruta.toFile());
    NodeList nodos = doc.getElementsByTagName("empleado");
    // HACIA ATRÁS: la NodeList es "viva" y encoge al borrar; hacia delante
te saltarías nodos
    for (int i = nodos.getLength() - 1; i >= 0; i--) {
        Element emp = (Element) nodos.item(i);
        String id = emp.getAttribute("id");
        if (id.equals("E002"))              // cambiar el texto de un hijo
            emp.getElementsByTagName("puesto").item(0).setTextContent("Jefa
de proyecto");
        if (id.equals("E001"))              // cambiar o añadir un atributo
            emp.setAttribute("activo", "false");
        if (id.equals("E003"))              // borrar el nodo entero
            emp.getParentNode().removeChild(emp);
    }
    Element nuevo = doc.createElement("empleado");   // añadir uno al final
de la raíz
    nuevo.setAttribute("id", "E004");
    Element n = doc.createElement("nombre"); n.setTextContent("Raúl Sanz");
    Element p = doc.createElement("puesto"); p.setTextContent("Becario");
```

```
    nuevo.appendChild(n); nuevo.appendChild(p);
    doc.getDocumentElement().appendChild(nuevo);

    guardar(doc, ruta);   // sin guardar, los cambios solo existen en memoria
}
```

Guardar sin líneas en blanco. Al reescribir un XML que ya tenía sangrías, el Transformer deja líneas vacías entre etiquetas (comprobado). Se arregla quitando los nodos de texto que solo contienen espacios antes de transformar:

```
static void limpiarBlancos(Node nodo) {
    NodeList hijos = nodo.getChildNodes();
    for (int i = hijos.getLength() - 1; i >= 0; i--) {
        Node h = hijos.item(i);
        if (h.getNodeType() == Node.TEXT_NODE &&
h.getTextContent().isBlank()) nodo.removeChild(h);
        else if (h.getNodeType() == Node.ELEMENT_NODE) limpiarBlancos(h);
    }
}

static void guardar(Document doc, Path ruta) throws TransformerException {
    limpiarBlancos(doc.getDocumentElement());
    TransformerFactory tf = TransformerFactory.newInstance();
    tf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
    Transformer t = tf.newTransformer();
    t.setOutputProperty(OutputKeys.INDENT, "yes");
    t.setOutputProperty(OutputKeys.ENCODING, "UTF-8");
    t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
    t.transform(new DOMSource(doc), new StreamResult(ruta.toFile()));
}
```

Buscar un nodo concreto es más corto con XPath: `Element e = (Element) xpath.evaluate("//empleado[@id='E002']", doc, XPathConstants.NODE);`

Tema 6.2: SAX y DOM vs SAX

SAX lee el XML de principio a fin y te avisa con eventos; no guarda nada en memoria. Solo lectura.

```
SAXParserFactory factory = SAXParserFactory.newInstance();
factory.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
factory.setFeature("http://apache.org/xml/features/disallow-doctype-decl",
true);
SAXParser parser = factory.newSAXParser();
```

```
DefaultHandler handler = new DefaultHandler() {
    private final StringBuilder texto = new StringBuilder();

    @Override public void startElement(String uri, String localName, String
qName, Attributes atts) {
        texto.setLength(0);                          // limpiar al abrir
etiqueta
        if (qName.equals("empleado")) System.out.println("id=" +
atts.getValue("id"));
    }
    @Override public void characters(char[] ch, int start, int length) {
        texto.append(ch, start, length);             // respetar start y
length; puede llegar en trozos
    }
    @Override public void endElement(String uri, String localName, String
qName) {
        if (qName.equals("nombre"))
System.out.println(texto.toString().trim());
    }
    @Override public void endDocument() { System.out.println("Fin"); }
};
parser.parse(new File("datos/empleados.xml"), handler);
```

	DOM	SAX
Cómo lee	Carga el árbol completo	Eventos secuenciales
Memoria	Alta (todo el documento)	Mínima
Modificar / crear XML	Sí	No, solo lectura
Volver atrás / acceso aleatorio	Sí	No
Clases	`DocumentBuilderFactory`, `Document`, `Element`, `NodeList`	`SAXParserFactory`, `SAXParser`, `DefaultHandler`
Cuándo	XML pequeño/mediano que vas a consultar o editar	XML enorme, lectura de una pasada, filtrar o contar

Tema 7 / 7.1 / 7.2: JAXP, validación XSD y XPath

JAXP es el paraguas estándar del JDK: todas sus piezas siguen el patrón `XxxFactory.newInstance()` → objeto de trabajo, y todas admiten `FEATURE_SECURE_PROCESSING`.

Fábrica	Paquete	Para qué
`DocumentBuilderFactory`	javax.xml.parsers	Parser DOM
`SAXParserFactory`	javax.xml.parsers	Parser SAX
`TransformerFactory`	javax.xml.transform	Escribir XML / XSLT
`SchemaFactory`	javax.xml.validation	Validar contra XSD
`XPathFactory`	javax.xml.xpath	Consultas XPath

Validar con XSD

```
SchemaFactory sf =
SchemaFactory.newInstance(XMLConstants.W3C_XML_SCHEMA_NS_URI);
sf.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD, "");
sf.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA, "");
Schema schema = sf.newSchema(new File("datos/clientes.xsd"));

DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setNamespaceAware(true);   // obligatorio para validar con XSD
dbf.setSchema(schema);
DocumentBuilder builder = dbf.newDocumentBuilder();
builder.setErrorHandler(new ErrorHandler() { ... });   // si no, los errores
solo se imprimen
Document doc = builder.parse(new File("datos/clientes.xml"));
```

XPath

```
XPathFactory xpf = XPathFactory.newInstance();
xpf.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true);
XPath xpath = xpf.newXPath();

// Un solo valor (2 argumentos → String)
String precio = xpath.evaluate("//producto[nombre='Monitor']/precio/text()",
doc);

// Varios nodos (3 argumentos + constante)
NodeList nombres = (NodeList) xpath.evaluate("//nombre", doc,
XPathConstants.NODESET);
for (int i = 0; i < nombres.getLength(); i++)
    System.out.println(nombres.item(i).getTextContent());

// Reutilizar: compilar una vez
XPathExpression expr = xpath.compile("//producto[precio>100]/@id");
NodeList ids = (NodeList) expr.evaluate(doc, XPathConstants.NODESET);
```

XPathConstants	Devuelve
`STRING` (o 2 argumentos)	String; si hay varios nodos, solo el primero sin avisar
`NUMBER`	Double (`count(//x)`, `sum(//precio)`)
`BOOLEAN`	Boolean
`NODE`	Un Node
`NODESET`	NodeList con todos

Expresión	Significa
`/a/b`	Ruta absoluta desde la raíz
`//b`	Todos los b, a cualquier profundidad
`@id`	Atributo
`text()`	Texto del nodo
`[precio>100]`	Filtro por valor
`[1]`, `[last()]`	Por posición (empieza en 1)
`count()`, `sum()`, `contains(.,'x')`, `not()`	Funciones

Expresión	Significa
`*`, `..`, `.`	Cualquier hijo, padre, actual

DOM vs XPath: con DOM recorres a mano con `getElementsByTagName` y bucles; con XPath escribes la consulta en una línea sobre el mismo `Document`.

Tema 8 / 8.1 / 8.2 / 8.3: JAXB, POJOs y Maven

JAXB convierte objetos Java a XML (marshal) y XML a objetos (unmarshal) usando anotaciones. Paquete actual: `jakarta.xml.bind` (no `javax.xml.bind`, no se mezclan).

pom.xml de JAXB (API + implementación)

```
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>4.0.2</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
  <version>4.0.5</version>
</dependency>
```

Sin `jaxb-runtime` falla al crear el `JAXBContext`: *Implementation of Jakarta XML Binding-API has not been found*.

POJO anotado para JAXB

```
import jakarta.xml.bind.annotation.*;

@XmlRootElement(name = "empleado")          // puede ser raíz del documento
@XmlAccessorType(XmlAccessType.FIELD)       // JAXB lee los campos, no
getters
@XmlType(propOrder = {"nombre", "departamento"}) // orden de los hijos
public class Empleado {
    @XmlAttribute private String id;                    // <empleado
id="...">
    @XmlElement   private String nombre;                // <nombre>...
</nombre>
    @XmlElement(name = "area") private String departamento; // renombra la
etiqueta
    @XmlTransient private String password;              // no sale en el XML
```

```
    public Empleado() {}                                    // ▲ OBLIGATORIO
constructor vacío: JAXB crea el objeto con él al hacer unmarshal.
                                                            // Si falta:
IllegalAnnotationsException ("does not have a no-arg default constructor")
                                                            // al crear el
JAXBContext, así que falla también al GUARDAR
    // getters/setters
}

@XmlRootElement(name = "empleados")
@XmlAccessorType(XmlAccessType.FIELD)
public class Empleados {
    @XmlElement(name = "empleado")          // nombra cada elemento de la
lista
    private List<Empleado> lista = new ArrayList<>();
    public Empleados() {}                  // ▲ OBLIGATORIO también en la
clase contenedora (y en cualquier clase anidada)
    public List<Empleado> getLista() { return lista; }
}
```

Marshal / unmarshal

```
JAXBContext ctx = JAXBContext.newInstance(Empleado.class);   // la clase raíz

Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);       // indentado
m.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");
m.marshal(emp, new File("datos/empleado.xml"));              // o
m.marshal(emp, System.out)

Unmarshaller u = ctx.createUnmarshaller();
Empleado leido = (Empleado) u.unmarshal(new File("datos/empleado.xml"));
```

Anotación	Efecto
`@XmlRootElement(name=)`	La clase puede ser la raíz; nombre de la etiqueta. Las clases anidadas no la necesitan
`@XmlAccessorType(FIELD)`	Mira campos (aunque sean private)
`@XmlElement(name=)`	Campo como etiqueta hija
`@XmlAttribute(name=)` vs `@XmlElement`	Atributo dentro de la etiqueta vs etiqueta hija
`@XmlTransient`	Excluido (y fuera del propOrder)

Anotación	Efecto
`@XmlType(propOrder=)`	Orden fijo de hijos

POJO para JAXB: clase normal sin herencia especial, campos privados, constructor sin argumentos, getters/setters. **JAXB vs DOM:** DOM construye nodo a nodo; JAXB mapea la clase entera en dos líneas.

Comandos Maven

```
mvn archetype:generate "-DgroupId=com.dam.xml" "-DartifactId=jaxb-maven-ejemplo"
mvn clean                    # borra target/
mvn -q clean package         # compila y empaqueta
mvn -q compile exec:java "-Dexec.mainClass=com.dam.xml.GuardarEmpleadoXML"
```

Tema 9: buenas prácticas (rutas, buffers, portabilidad)

Práctica	Mal	Bien
Rutas relativas y portables	`"C:\\Users\\yo\\datos\\a.txt"` o `"datos" + "/" + f`	`Path.of("datos", "a.txt")` (pone el separador del SO)
Crear carpeta antes	`new FileWriter("datos/salida.txt")` sin carpeta → `FileNotFoundException`	`Files.createDirectories(ruta.getParent())`
Charset explícito	`new FileReader(f)`, `new Scanner(new File(f))`, `new String(bytes)`	`new FileReader(f, UTF_8)`, `new Scanner(f, UTF_8)`, `new String(bytes, UTF_8)`
Buffer	FileReader/FileWriter a pelo	Envolver en BufferedReader/BufferedWriter
Cerrar	`close()` manual (no se ejecuta si salta excepción antes)	try-with-resources
Saltos de línea	`"\n"` a mano	`newLine()` o `System.lineSeparator()`
Excepciones	`catch (Exception e)`	Concreta primero (`NoSuchFileException`) y luego `IOException`

Las rutas relativas se resuelven contra el directorio de trabajo (donde se ejecuta el programa; en IntelliJ, la raíz del proyecto). Para comprobarlo: `Path.of("").toAbsolutePath()`.

Tema 10: probar y documentar (JUnit 5 + Javadoc)

Probar es escribir código que comprueba tu código (JUnit 5); documentar es explicar qué promete cada método para usarlo sin leerlo (Javadoc). Todo lo de esta sección está comprobado en JDK 25: los 7 tests pasan y el Javadoc no da avisos ni con `doclint` completo.

pom.xml para pruebas y Javadoc

```
<properties>
  <maven.compiler.release>25</maven.compiler.release>          <!-- versión
de Java (source + target) -->
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding> <!--
codificación de los .java -->
</properties>
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>   <!-- API + motor de una vez -->
    <version>5.14.4</version>
    <scope>test</scope>                       <!-- solo al probar, no va en
el .jar -->
  </dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>  <!-- ejecuta los tests
en mvn test -->
      <version>3.6.0</version>
    </plugin>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-javadoc-plugin</artifactId>   <!-- genera la
documentación -->
      <version>3.12.0</version>
      <configuration>
        <encoding>UTF-8</encoding>       <!-- lee los .java -->
        <docencoding>UTF-8</docencoding> <!-- escribe los HTML -->
        <charset>UTF-8</charset>         <!-- meta charset del HTML -->
        <!-- opcional: <doclint>all</doclint>
<failOnWarnings>true</failOnWarnings> -->
```

```
      </configuration>
    </plugin>
  </plugins>
</build>
```

Estructura: el código en `src/main/java/com/dam/ficheros/FicheroDAO.java` y la prueba en `src/test/java/com/dam/ficheros/FicheroDAOTest.java`. Mismo paquete en los dos sitios, y la clase de prueba termina en `Test` (así la encuentra Surefire). Los imports de JUnit 5 son `org.junit.jupiter.api.*`; si ves `org.junit.Test`, `@Before` o `Assert.assertEquals`, es JUnit 4 y no se mezcla.

Clase documentada con Javadoc

```
/**
 * Acceso de lectura y escritura a ficheros de texto plano.
 *
 * <p>Todas las operaciones usan {@link StandardCharsets#UTF_8} de forma
explícita.
 * Las carpetas intermedias se crean automáticamente al escribir.</p>
 *
 * <pre>{@code
 * FicheroDAO dao = new FicheroDAO(Path.of("datos"));
 * dao.guardarLineas("alumnos.txt", List.of("Ana", "Íñigo"));
 * }</pre>
 *
 * @author Sebas
 * @version 1.0
 */
public class FicheroDAO {
    /** Carpeta base sobre la que se resuelven los nombres de fichero. */
    private final Path base;

    /**
     * Crea un DAO que trabaja sobre la carpeta indicada.
     *
     * @param base carpeta base; no puede ser {@code null}
     * @throws NullPointerException si {@code base} es {@code null}
     */
    public FicheroDAO(Path base) {
        this.base = Objects.requireNonNull(base, "La carpeta base no puede
ser null");
    }

    /**
     * Escribe las líneas en un fichero, sobrescribiendo lo que hubiera.
```

```
     * Crea la carpeta base y las intermedias si no existen.
     *
     * @param nombre nombre del fichero, relativo a la carpeta base
     * @param lineas líneas a escribir, en orden
     * @return la ruta del fichero escrito
     * @throws IOException si falla la escritura en disco
     */
    public Path guardarLineas(String nombre, List<String> lineas) throws
IOException {
        Path destino = base.resolve(nombre);          // datos + alumnos.txt
        Files.createDirectories(destino.getParent());
        Files.write(destino, lineas, StandardCharsets.UTF_8);
        return destino;
    }

    /**
     * Lee todas las líneas de un fichero de texto en UTF-8.
     *
     * @param nombre nombre del fichero, relativo a la carpeta base
     * @return lista con las líneas; lista vacía (nunca {@code null}) si el
fichero está vacío
     * @throws NoSuchFileException si el fichero no existe
     * @throws IOException si falla la lectura por cualquier otro motivo
     */
    public List<String> leerLineas(String nombre) throws IOException {
        return Files.readAllLines(base.resolve(nombre),
StandardCharsets.UTF_8);
    }

    /**
     * Cuenta las líneas no vacías (las que solo tienen espacios no cuentan).
     *
     * @param nombre nombre del fichero, relativo a la carpeta base
     * @return número de líneas con contenido
     * @throws IOException si falla la lectura
     */
    public long contarLineasConContenido(String nombre) throws IOException {
        return leerLineas(nombre).stream().filter(l -> !l.isBlank()).count();
    }

    /**
     * Indica si existe un fichero con ese nombre dentro de la carpeta base.
     *
     * @param nombre nombre del fichero, relativo a la carpeta base
     * @return {@code true} si el fichero existe
     */
```

```
    public boolean existe(String nombre) {
        return Files.exists(base.resolve(nombre));
    }
}
```

Etiqueta	Para qué	Dónde
Primera frase	Resumen que sale en la tabla de métodos (acabar en punto)	Clase, método, campo
`@param nombre desc`	Un parámetro; uno por cada uno, en orden	Método, constructor
`@return desc`	Qué devuelve (no en `void`)	Método
`@throws Tipo desc`	Cuándo lanza esa excepción (comprobada o no)	Método, constructor
`@author`, `@version`	Autoría y versión	Clase
`{@link Clase#metodo}`	Enlace a otro elemento	Cualquier texto
`{@code texto}`	Texto en monoespaciado	Cualquier texto
`<pre>{@code ... }</pre>`	Bloque de código de ejemplo	Cualquier texto

Javadoc empieza por `/**` (dos asteriscos) y va justo encima de lo que documenta. `@throws` documenta cualquier excepción; la cláusula `throws` de la firma solo lleva las comprobadas (por eso el constructor documenta `NullPointerException` sin declararla). Un buen Javadoc explica lo que no se ve en la firma: qué pasa si no existe, qué codificación usa, si puede devolver `null`.

Clase de prueba con JUnit 5

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;
import static org.junit.jupiter.api.Assertions.*;   // assertEquals sin
"Assertions." delante

@DisplayName("FicheroDAO")
class FicheroDAOTest {

    @TempDir
    Path carpetaTemporal;        // JUnit la crea antes de cada test y la
```

```
borra después

    FicheroDAO dao;

    @BeforeEach
    void preparar() {            // se ejecuta antes de CADA test
        dao = new FicheroDAO(carpetaTemporal);
    }

    @Test
    @DisplayName("guarda y recupera las mismas líneas")
    void guardaYRecupera() throws IOException {
        List<String> original = List.of("Primera línea", "Segunda línea");
// Preparar
        dao.guardarLineas("prueba.txt", original);
// Actuar
        assertEquals(original, dao.leerLineas("prueba.txt"));
// Comprobar
    }

    @Test
    @DisplayName("conserva tildes y eñes (se escribe en UTF-8)")
    void conservaAcentos() throws IOException {
        String texto = "Íñigo Muñoz, año 2026, cigüeña";
        Path destino = dao.guardarLineas("acentos.txt", List.of(texto));
        assertEquals(texto, dao.leerLineas("acentos.txt").get(0));
        byte[] bytes = Files.readAllBytes(destino);
// mirar los BYTES
        assertArrayEquals("Í".getBytes(StandardCharsets.UTF_8),
// Í = 2 bytes en UTF-8
                new byte[]{bytes[0], bytes[1]}, "El fichero no se ha escrito
en UTF-8");
    }

    @Test
    @DisplayName("un fichero que no existe lanza NoSuchFileException")
    void ficheroInexistente() {
        NoSuchFileException ex = assertThrows(NoSuchFileException.class,
                () -> dao.leerLineas("no_existe.txt"));
        assertTrue(ex.getFile().endsWith("no_existe.txt"));
    }

    @Test
    @DisplayName("un fichero vacío devuelve una lista vacía, no null")
    void ficheroVacio() throws IOException {
        dao.guardarLineas("vacio.txt", List.of());
```

```
        assertNotNull(dao.leerLineas("vacio.txt"));
        assertTrue(dao.leerLineas("vacio.txt").isEmpty());
        assertEquals(0, dao.contarLineasConContenido("vacio.txt"));
    }

    @Test
    @DisplayName("crea las carpetas intermedias que falten")
    void creaCarpetasIntermedias() throws IOException {
        FicheroDAO dao2 = new
FicheroDAO(carpetaTemporal.resolve("datos").resolve("2026"));
        assertFalse(dao2.existe("informe.txt"));
        dao2.guardarLineas("informe.txt", List.of("ok"));
        assertTrue(dao2.existe("informe.txt"));
    }

    @Test
    @DisplayName("cuenta solo las líneas con contenido")
    void cuentaLineasConContenido() throws IOException {
        dao.guardarLineas("mixto.txt", List.of("uno", "", "   ", "dos"));
        assertEquals(2, dao.contarLineasConContenido("mixto.txt"));
    }

    @Test
    @DisplayName("la carpeta base no puede ser null")
    void baseNoPuedeSerNull() {
        assertThrows(NullPointerException.class, () -> new FicheroDAO(null));
    }
}
```

Anotación	Qué hace
`@Test`	Marca un método como prueba (`void` , sin parámetros, no `static`)
`@DisplayName("...")`	Nombre legible en el informe; escribe qué debe pasar
`@TempDir`	Carpeta temporal nueva por test, borrada al terminar: no ensucia y el orden da igual
`@BeforeEach` / `@AfterEach`	Antes / después de cada test
`@BeforeAll` / `@AfterAll`	Una vez para toda la clase; tienen que ser `static`
`@Disabled("motivo")`	Desactiva un test sin borrarlo (sale como skipped)

Aserción	Comprueba
`assertEquals(esperado, real)`	Que son iguales; el esperado va primero
`assertTrue(cond)` / `assertFalse(cond)`	Una condición
`assertNotNull(obj)` / `assertNull(obj)`	Que (no) es null
`assertArrayEquals(esp, real, "mensaje")`	Dos arrays, con mensaje si falla
`assertThrows(Tipo.class, () -> ...)`	Que lanza esa excepción; la devuelve para inspeccionarla

Patrón AAA en cada test: **Preparar** (crear objetos y datos), **Actuar** (llamar al método), **Comprobar** (aserciones). Un método que solo hace `System.out.println` no se puede probar: saca la lógica a métodos que devuelvan valores.

Ejecutar pruebas y generar la documentación

```
mvn test               # ejecuta los tests → informes en target/surefire-reports/
mvn javadoc:javadoc    # documentación → target/reports/apidocs/index.html
                       # (plugins antiguos: target/site/apidocs/)
```

Salida correcta: `Tests run: 7, Failures: 0, Errors: 0, Skipped: 0` y `BUILD SUCCESS`.

Demostrar que las pruebas sirven: cambia en `guardarLineas` el Charset por `Charset.forName("windows-1252")` (con `import java.nio.charset.Charset;`). Fallan 2 tests con `MalformedInputException: Input length = 1` y pasan los otros 5 (comprobado en JDK 25). Quitar el Charset sin más no rompe nada en JDK 25, porque el defecto ya es UTF-8.

Plan de pruebas y qué documentar

#	Caso	Por qué importa	Test del ejemplo
1	Camino feliz	Que lo normal funciona	`guardaYRecupera`
2	Tildes y eñes	Errores de codificación invisibles en pantalla	`conservaAcentos`
3	El fichero no existe	El error más frecuente	`ficheroInexistente`

#	Caso	Por qué importa	Test del ejemplo
4	Fichero vacío	¿Lista vacía o `null`?	`ficheroVacio`
5	La carpeta no existe	El fallo número 1 del RA	`creaCarpetasIntermedias`
6	Casos límite	Líneas en blanco, espacios, extremos	`cuentaLineasConContenido`
7	Argumentos inválidos	`null`, cadenas vacías, negativos	`baseNoPuedeSerNull`

Regla práctica: por cada método público, un test del caso normal y al menos uno del caso que falla.

Nivel	Qué se documenta	Herramienta
Clase	Qué representa, cómo se usa, qué garantiza	Javadoc `/** */`
Método público	Parámetros, retorno, excepciones, efectos	`@param`, `@return`, `@throws`
Dentro del método	Solo el porqué de una decisión no obvia	`//`
Proyecto	Cómo compilar, ejecutar y probar	`README.md`

Comprobación final del RA1 (todo programa debería cumplirla): crea el directorio antes de escribir · Charset en todas las lecturas y escrituras · try-with-resources · captura la excepción concreta con mensaje útil · XML con declaración de codificación y parsers endurecidos · pruebas automáticas que pasan · Javadoc en los métodos públicos.

Tema 12: conversiones entre formatos

Convertir = leer un formato a objetos (el modelo, p. ej. `List<Alumno>`) y escribir esos objetos en otro. Con n formatos haces 2n métodos (un lector y un escritor por formato), no n×(n−1) conversores directos. Se comprueba con ida y vuelta: los objetos leídos deben ser iguales a los originales.

	CSV (texto)	Binario (DataOutputStream)	Serializado (ObjectOutputStream)	XML
Legible por persona	Sí	No	No	Sí
Lo lee otro lenguaje	Sí	Sí, con esfuerzo	No, solo Java	Sí
Guarda tipos	No, todo es texto	Sí	Sí, y la clase entera	No (salvo con XSD)
Tamaño (5 alumnos)	177 bytes	192 bytes	388 bytes	873 bytes
Estructura	Plana	Plana	La del objeto	Jerárquica
Depende de codificación	Sí, y no la declara	No (writeUTF la fija)	No	Sí, y la declara

CSV: leer bien

```
List<String> lineas = Files.readAllLines(ruta, StandardCharsets.UTF_8);
for (String l : lineas.subList(1, lineas.size())) {     // saltar cabecera
    String[] c = l.split(";", -1);                       // -1: conserva
vacíos del final
    lista.add(new Alumno(Integer.parseInt(c[0]), c[1],
Double.parseDouble(c[2])));
}
// "a;b;;".split(";")     → 2 elementos
// "a;b;;".split(";", -1) → 4 elementos
```

Si un campo puede llevar `;` o comillas, `split` no basta: se recorre carácter a carácter con un flag `entrecomillado` y `""` como comilla escapada.

CSV: escribir bien

```
StringBuilder sb = new StringBuilder("id;nombre;nota\n");
for (Alumno a : lista)
sb.append(a.id()).append(';').append(escapar(a.nombre())).append(';').append(
a.nota()).append('\n');
Files.writeString(ruta, sb.toString(), StandardCharsets.UTF_8);
// escapar: si contiene ; o " → "..." y duplicar las comillas internas
```

Escapado por formato: CSV lo haces tú (entrecomillar); XML lo hace la librería (`<` , `&`); JSON usa `\"` y `\\` (no viene en el JDK, sí con Jackson). Escapar dos veces es tan malo como no escapar. JSON siempre es UTF-8 (RFC 8259).

Extra: JSON con Jackson (no está en los PDF)

Los temas no traen código JSON: el Tema 12 solo dice que el JDK no lo incluye y que se usa Jackson o Gson. Jackson funciona igual que JAXB: anotas un POJO y un objeto (`ObjectMapper`) lo convierte a JSON y de vuelta.

pom.xml de Jackson

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
  <version>2.18.2</version>   <!-- trae core y annotations -->
</dependency>
```

POJO para Jackson

```
import com.fasterxml.jackson.annotation.*;

@JsonPropertyOrder({"id", "nombre", "nota"})   // orden de las claves (como
propOrder)
public class Alumno {
    private int id;
    @JsonProperty("nombre_completo")           // renombra la clave (como
@XmlElement(name=))
    private String nombre;
    private double nota;
    @JsonIgnore private String password;        // no sale (como
@XmlTransient)

    public Alumno() {}                          // ⚠ OBLIGATORIO constructor
vacío: Jackson lo usa en readValue.
                                                // Si falta:
MismatchedInputException "Cannot construct instance"
    public Alumno(int id, String nombre, double nota) { ... }
    // getters y setters: Jackson lee/escribe a través de ellos
}
```

Escribir y leer JSON (objeto ↔ fichero)

```
ObjectMapper mapper = new ObjectMapper();
mapper.enable(SerializationFeature.INDENT_OUTPUT);       // JSON indentado
(como JAXB_FORMATTED_OUTPUT)
Files.createDirectories(Path.of("datos"));

// Objeto → JSON
mapper.writeValue(Path.of("datos", "alumno.json").toFile(), alumno);

// JSON → objeto
Alumno a = mapper.readValue(Path.of("datos", "alumno.json").toFile(),
Alumno.class);

// Lista → JSON y JSON → lista (TypeReference porque List<Alumno>.class no
existe)
mapper.writeValue(Path.of("datos", "alumnos.json").toFile(), lista);
List<Alumno> leidos = mapper.readValue(Path.of("datos",
"alumnos.json").toFile(),
                                       new TypeReference<List<Alumno>>() {});

// A / desde String
String json = mapper.writeValueAsString(alumno);
Alumno b = mapper.readValue(json, Alumno.class);
```

De JSON a objeto paso a paso (deserializar)

Pasar de objeto a JSON es serializar; de JSON a objeto es deserializar. Este es el fichero `datos/curso.json`, con un objeto dentro de otro y una lista:

```
{
  "nombre": "DAM 2",
  "tutor": { "nombre": "Laura", "email": "laura@centro.es" },
  "alumnos": [
    { "id": 1, "nombre_completo": "Ana Torres", "nota": 8.5 },
    { "id": 2, "nombre_completo": "Luis Pérez", "nota": 6.0 }
  ]
}
```

Una clase por cada `{ }` y un `List` por cada `[ ]`, con los mismos nombres de clave:

```
@JsonIgnoreProperties(ignoreUnknown = true)   // si el JSON trae claves que
la clase no tiene, las ignora
public class Curso {
    private String nombre;
    private Tutor tutor;                       // { } anidado → otra clase
    private List<Alumno> alumnos;              // [ ] → List
    public Curso() {}                          // ⚠ OBLIGATORIO constructor
vacío
    // getters y setters
}

public class Tutor {
    private String nombre;
    private String email;
    public Tutor() {}                          // ⚠ OBLIGATORIO también en
las clases anidadas
    // getters y setters
}
// Alumno es la clase de arriba (con @JsonProperty("nombre_completo"))
```

```
ObjectMapper mapper = new ObjectMapper();
Curso curso = mapper.readValue(Path.of("datos", "curso.json").toFile(),
Curso.class);

System.out.println(curso.getNombre());                    // DAM 2
System.out.println(curso.getTutor().getEmail());          // laura@centro.es
for (Alumno a : curso.getAlumnos())
    System.out.println(a.getNombre() + " → " + a.getNota());
```

Qué hace Jackson por dentro en `readValue`:

1. Crea un `Curso` vacío con el constructor sin argumentos (si no existe: `MismatchedInputException`).
2. Recorre las claves del JSON y busca para cada una el setter con el mismo nombre (`"nombre"` → `setNombre`) o el nombre puesto en `@JsonProperty`.
3. Convierte el valor al tipo del campo: `8.5` → `double`, `"DAM 2"` → `String`, `{ }` → un `Tutor` nuevo (vuelve al paso 1), `[ ]` → un `ArrayList` de `Alumno`.
4. Las claves que faltan en el JSON se quedan con su valor por defecto (`null`, `0`, `false`).

Error al leer	Causa	Solución
`UnrecognizedPropertyException`	El JSON trae una clave que la clase no tiene	`@JsonIgnoreProperties(ignoreUnknown = true)` o `mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false)`
`MismatchedInputException: Cannot construct instance … no Creator`	Falta el constructor vacío	Añadirlo (o `@JsonCreator` en el constructor con argumentos)
`InvalidFormatException`	El valor no se puede convertir: `"8,5"` (texto con coma) en un `double`	Corregir el JSON (`8.5`)
`MismatchedInputException`	La forma no cuadra: un `[ ]` leído con `Alumno.class`	Leer con `TypeReference<List<...>>`
`JsonParseException`	JSON mal escrito (coma sobrante, comillas simples)	Revisar el fichero

Todas son subclases de `IOException` (comprobado), así que con `catch (IOException e)` las capturas todas.

Leer varios: JSON que empieza por [→ ArrayList<Alumno>

`readValue` lo hace todo en una línea (abre, lee, crea los objetos, rellena anidados y listas, cierra). Pero si el fichero es una lista suelta `[ {...}, {...} ]`, con `Alumno.class` lanza `MismatchedInputException`. Tres formas:

```
File f = Path.of("datos", "alumnos.json").toFile();

// 1) TypeReference (la más habitual)
ArrayList<Alumno> arrayAlumno = mapper.readValue(f, new
TypeReference<ArrayList<Alumno>>() {});

// 2) Como array y luego a ArrayList
ArrayList<Alumno> arrayAlumno = new ArrayList<>
(Arrays.asList(mapper.readValue(f, Alumno[].class)));

// 3) Indicando el tipo con la TypeFactory
ArrayList<Alumno> arrayAlumno = mapper.readValue(f,
        mapper.getTypeFactory().constructCollectionType(ArrayList.class,
Alumno.class));
```

- **Por qué `TypeReference`:** Java borra el genérico al compilar, así que `ArrayList<Alumno>.class` no existe. Sin él Jackson no sabe qué va dentro y devuelve una lista de `LinkedHashMap`.
- **Por qué las `{}` del final:** crean una clase anónima, que es lo que conserva el `<Alumno>`. Sin ellas no compila.
- **`List` o `ArrayList`:** si la variable es `List<Alumno>`, usa `TypeReference<List<Alumno>>`; Jackson devuelve igualmente un `ArrayList`.
- **Lista dentro de un objeto** (como `alumnos` en `curso.json`): no hace falta nada de esto. Lees con `Curso.class` y Jackson ve el tipo en el campo `List<Alumno> alumnos`.

Terminología: objeto → JSON (`writeValue`) es **serializar**; JSON → objeto (`readValue`) es **deserializar**.

Leer JSON sin clase (árbol, como DOM)

```
JsonNode raiz = mapper.readTree(Path.of("datos", "alumnos.json").toFile());
for (JsonNode n : raiz) {                                  // si la raíz es un
array
    System.out.println(n.get("nombre_completo").asText() + " " +
n.get("nota").asDouble());
}
```

Conversión CSV → JSON

```
List<Alumno> lista = leerCsv(Path.of("datos", "alumnos.csv"));           //
lector del Tema 12
new ObjectMapper().enable(SerializationFeature.INDENT_OUTPUT)
                  .writeValue(Path.of("datos", "alumnos.json").toFile(),
lista);
// XML → JSON: unmarshal con JAXB a objetos y después mapper.writeValue(...)
```

Jackson	Para qué	Equivalente JAXB
`ObjectMapper`	Hace todo: escribir y leer	`JAXBContext` + `Marshaller` / `Unmarshaller`
`writeValue(file, obj)`	Objeto → fichero JSON	`marshal(obj, file)`
`readValue(file, Clase.class)`	Fichero JSON → objeto	`unmarshal(file)` + cast
`writeValueAsString` / `readValue(String, ...)`	Objeto ↔ String	`marshal(obj, StringWriter)`
`TypeReference<List<T>>`	Leer listas genéricas	Clase contenedora con `List`
`readTree` → `JsonNode`	Recorrer sin clase	DOM
`@JsonProperty`, `@JsonIgnore`, `@JsonPropertyOrder`	Renombrar, excluir, ordenar	`@XmlElement(name)`, `@XmlTransient`, `@XmlType(propOrder)`
`INDENT_OUTPUT`	Salida legible	`JAXB_FORMATTED_OUTPUT`

Diferencias JSON vs XML: JSON no tiene atributos (todo son claves), es siempre UTF-8 (no se declara), distingue número/texto/booleano/null y ocupa menos. Jackson escapa solo `\"` y `\\`: no escapes tú antes. Fechas `LocalDate`: hace falta el módulo `jackson-datatype-jsr310` y `mapper.registerModule(new JavaTimeModule())`.